LAB REPORT

OWASP A07:2021

Identification and Authentication Failures

Broken Authentication & Session Management — Brute Force, Delay-Bypass, Session Fixation, and Weak Session ID Generation on DVWA

Field	Detail
Target Application	DVWA (Damn Vulnerable Web Application) v1.10 *Development*
Deployment	Docker (vulnerables/web-dvwa), localhost:8080
Host OS	Windows 11
Attacker Tooling Environment	Kali Linux (via WSL2)
Security Levels Tested	Low, Medium, High (source-level comparison)
Primary Tools	Hydra v9.7 (THC-Hydra), Chrome DevTools, DVWA source review
Report Date	September 6, 2026
Category Scope	OWASP Top 10 (2021) — A07: Identification and Authentication Failures

Scope note: All activity in this report was performed exclusively against a local, self-hosted, intentionally vulnerable instance of DVWA running on **localhost**, inside an isolated lab environment owned and controlled by the author. No external systems were accessed or targeted at any point.

Table of Contents

1. Executive Summary

This report documents a hands-on lab session covering **OWASP A07:2021 — Identification and Authentication Failures** against DVWA. The session progressed through four connected practical exercises: (1) manual reconnaissance and automated credential brute forcing of a GET-based login form using Hydra, (2) demonstrating that a server-side sleep() delay is an insufficient standalone defense against brute force due to Hydra's multi-threaded attack model, (3) a full end-to-end **session fixation** attack proving that an unauthenticated session identifier can be silently escalated to an authenticated one, and (4) a source-code-level comparison of DVWA's four session ID generation strategies, establishing why hashing a predictable value does not produce a secure token.

Every exploit in this report was executed and independently verified by the author on their own local lab instance — none of the findings are theoretical.

Session Outcome Summary

Exercise	Result	Notes
Credential brute force (Hydra, Low)	SUCCESS	Cracked admin password ("password") from rockyou.txt in seconds
Delay-based defense (Medium, sleep())	BYPASSED	16 parallel Hydra threads absorbed the per-request delay
Session fixation (cross-browser PoC)	SUCCESS	Pre-login session ID became authenticated after victim login — zero credentials used
Weak Session ID review (source code)	COMPLETE	Low/Medium/High all shown to derive from predictable input

2. Objectives

- Understand and reproduce the core weaknesses that fall under OWASP A07 (weak credential policy, absence of rate limiting/lockout, session fixation, and predictable session token generation).
- Perform a real automated brute-force attack using Hydra against a live login form and correctly interpret its output and error conditions.
- Evaluate whether a rate-limiting delay (sleep()) is an effective standalone control against a multi-threaded attacker.
- Demonstrate — hands-on, not just conceptually — how a session fixation attack allows session hijacking without ever touching a username or password.
- Read and compare real backend PHP source code across four security tiers to understand *why* certain session ID generation strategies are cryptographically weak, even when they superficially resemble secure tokens (e.g., an MD5 hash).

3. Lab Environment

Component	Detail
Target application	DVWA (Damn Vulnerable Web Application) v1.10 *Development*
Deployment method	Docker container (image: vulnerables/web-dvwa)
Target address	http://localhost:8080
Host operating system	Windows 11
Attack tooling environment	Kali Linux via WSL2
Primary attack tool	Hydra v9.7 (c) 2023, van Hauser/THC & David Maciejak
Wordlist used	/usr/share/wordlists/rockyou.txt
Browser / inspection tooling	Google Chrome + DevTools (Application → Cookies), Incognito window
Security levels covered	Low (full exploit), Medium (delay bypass), High & Impossible (source review only)

4. Theory — What A07 Covers

A07:2021 — Identification and Authentication Failures (formerly named "Broken Authentication" in the 2017 edition) covers weaknesses in how an application verifies *who a user is* and *proves* that identity across a session. Unlike injection-class vulnerabilities, these flaws frequently require no code-level bug at all — a weak policy plus the absence of a specific control is often sufficient to compromise an account.

Core Attack Categories Under A07

Category	Description
Weak password policy	No minimum length/complexity, no breached-password check, permits defaults (e.g. admin/password)
Brute force / credential stuffing	Repeated login attempts against one account, or replaying leaked credential pairs, with no lockout or rate limiting to stop it
Session management flaws	Session fixation, session ID exposed in URLs, no timeout, no regeneration on privilege change
Missing multi-factor authentication	A single leaked password is sufficient for full account takeover
Improper credential recovery	Guessable security questions, predictable or non-expiring password-reset tokens

Decision Rule Used Throughout This Lab

A flaw is classified as A07 when it lives at the **authentication/session boundary itself** — i.e. it lets an attacker either *become* authenticated or *impersonate* an already-authenticated session, without exploiting a data-access flaw after the fact. Exploiting what an already-logged-in user can reach (e.g. changing another user's record ID in a URL) is classified separately, under A01 — Broken Access Control.

5. Methodology & Step-by-Step Walkthrough

5.1 Reconnaissance of the Brute Force Login Form

Before any automation, the DVWA "Brute Force" module's login form was inspected manually via View Source. The relevant markup:

```
<form action="#" method="GET">
Username:<br />
<input type="text" name="username"><br />
Password:<br />
<input type="password" AUTOCOMPLETE="off" name="password"><br />
<input type="submit" value="Login" name="Login">
</form>
```

KEY FINDING — The form uses **method="GET"**, meaning submitted credentials are appended directly to the URL rather than sent in a POST body. This exposes credentials in browser history and any server access logs — a secondary weakness beyond the brute-forceability itself. In addition, DVWA's own system-info panel at Low security discloses the correct username outright (Username: admin), reducing the attack surface to the password alone.

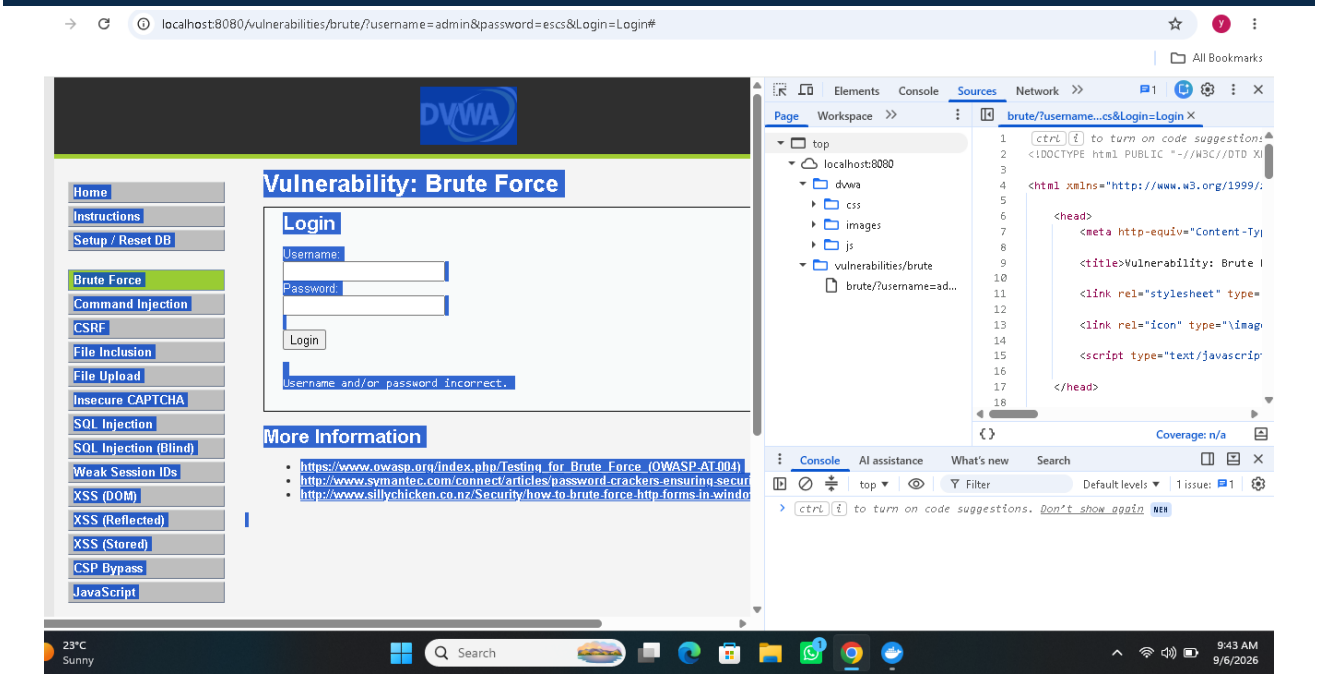

Figure 1 — DVWA Brute Force (Low) page source. The GET-method form and the disclosed correct username ("admin") are visible in the bottom-left system-info panel.

A manual wrong-password submission confirmed the resulting URL structure, which was then used to build the Hydra attack:

```
http://localhost:8080/vulnerabilities/brute/?username=admin&password=gmhghmdgh&Login=Login#
```

The exact failure text returned by the application — *"Username and/or password incorrect."* — was also recorded, as Hydra requires this string to distinguish a failed attempt from a successful one.

5.2 Building the Hydra Attack (Low Security)

Two prerequisites were gathered from the browser before constructing the command:

- A valid **PHPSESSID** cookie value (DVWA requires an authenticated-to-the-app session cookie just to reach the vulnerable page, so Hydra must present one too).
- The **security** cookie value (low), which DVWA also reads to determine which code path serves the request.

Initial (first-attempt) command:

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost -s 8080 http-get-form \
"/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:F=incorrect:H=Cookie:
security=low; PHPSESSID=<value>"
```

5.3 Troubleshooting Hydra Syntax Errors

The initial command failed. Two separate syntax issues had to be diagnosed and corrected in turn — both stemming from how Hydra's http-get-form module parses its colon-delimited argument string.

#	Error Observed	Root Cause	Fix Applied
1	no valid optional parameter type given: F	The literal colon inside "Cookie: security=low" was being read as a new field delimiter, corrupting the parser's view of the string.	Escaped the colon: changed H=Cookie: to H=Cookie\:
2	no valid optional parameter type given: F (persisted)	The H= (header) field was placed after the F= (failure-string) field in the argument order, which Hydra's http-get-form module does not parse correctly in that sequence.	Reordered the fields so H=Cookie\:... appears before F=incorrect at the end of the string

Corrected, working command:

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost -s 8080 http-get-form \
"/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie\: security=low;
PHPSESSID=sen54e32uagpgf69huavv3gbs0:F=incorrect"
```

KEY FINDING — Debugging tool syntax errors is itself a practical skill this exercise reinforced: reading Hydra's own error output, isolating which field it referred to, and iterating on argument order/escaping rather than abandoning the tool.

5.4 Successful Credential Crack

With the corrected syntax, Hydra completed the attack and returned a valid credential pair within seconds (the correct password appears early in rockyou.txt):

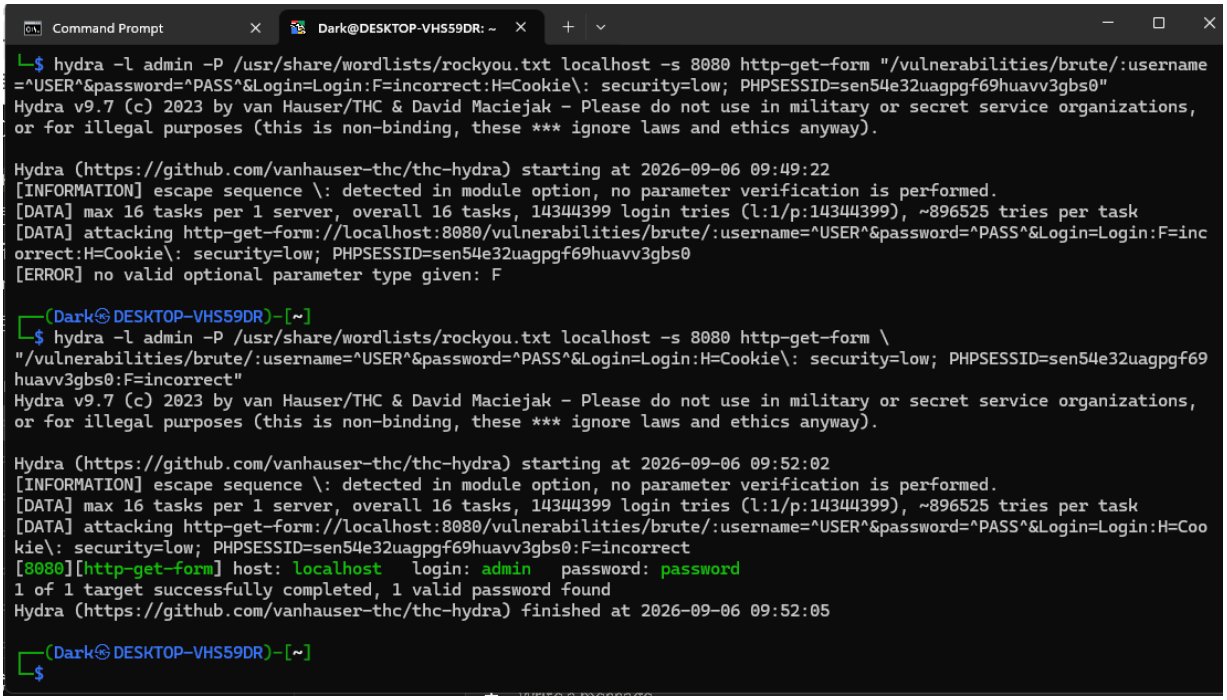

Figure 2 — Terminal output showing the initial syntax failure, the corrected command, and Hydra's successful result line: admin / password.

```
[8080][http-get-form] host: localhost login: admin password: password
1 of 1 target successfully completed, 1 valid password found
```

The result was manually verified by logging into DVWA through the browser with the recovered credentials, confirming the crack was genuine and not a false positive.

5.5 Medium Security — Defeating the sleep() Delay

DVWA's Medium security level adds a server-side sleep() delay after each failed login attempt, intended to slow brute-force attacks. The same Hydra command was re-run with the cookie's security value changed to medium:

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost -s 8080 http-get-form \
"/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie\: security=medium;
PHPSESSID=sen54e32uagpgf69huavv3gbs0:F=incorrect"
```

Result: the correct password was recovered almost immediately, with no perceptible slowdown compared to the Low-security run.

> **KEY FINDING —** Hydra defaults to **16 parallel connections**. A per-request sleep() only delays each individual thread — it does not throttle the attacker's aggregate request rate, since all 16 threads are delayed *simultaneously*, not sequentially. Because the correct password sits early in the wordlist, one of the sixteen parallel threads reached it before the cumulative delay became noticeable. A delay-only defense provides no meaningful protection against a threaded or distributed attacker.

This was cross-referenced against the earlier discovery (from a prior CSRF lab) that DVWA's **High** security level adds a per-request anti-CSRF token to the login form — a control that would actually stop Hydra's static request template, unlike the Medium-level delay.

Effective Layered Defenses Identified

- **Account lockout** after N consecutive failed attempts, independent of thread/connection count.
- **IP-based rate limiting** applied globally per source, not per individual request thread.
- **CAPTCHA** triggered after a small number of failures.
- All three layered together, rather than relying on any single control in isolation.

5.6 Session Fixation — End-to-End Proof of Concept

Before testing the exploit, session behavior was checked directly: the PHPSESSID cookie value was compared before and after a successful login through the normal browser form. The value did **not change** — confirming DVWA does not regenerate the session identifier at authentication time. This is the precondition required for a session fixation attack.

Attack Steps Performed

Step 1. Opened an Incognito ("victim") browser window at the DVWA login page *without logging in*, and captured its pre-authentication PHPSESSID (referred to below as SESSION_X).

Step 2. In a separate ("attacker") browser, manually overwrote the local PHPSESSID cookie value to match SESSION_X via DevTools, then refreshed — forcing the attacker's browser onto the same, still-unauthenticated session.

Step 3. In the victim (incognito) window, logged in normally with valid admin credentials.

Step 4. Returned to the attacker browser (still holding SESSION_X, never having submitted any login form) and refreshed the DVWA dashboard.

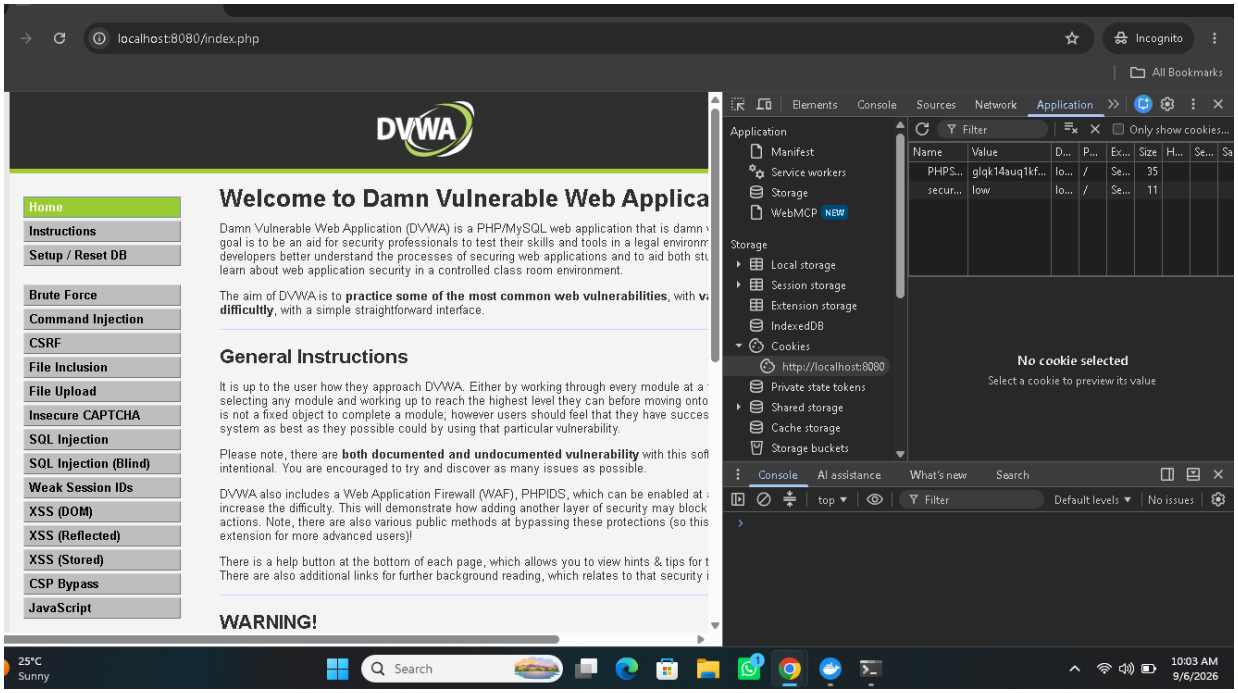

Figure 3 — The unauthenticated ("attacker") browser, holding only the pre-login session ID fixed onto it beforehand, is redirected to index.php and rendered the authenticated DVWA dashboard immediately after the victim's login in a separate window — with zero credentials ever entered in this browser.

KEY FINDING — The attacker's browser became fully authenticated the moment the victim logged in on a completely different browser session — no password, no cookie theft, and no interaction with the victim's machine beyond having pre-set a known session ID. This is the defining mechanism of a **session fixation** vulnerability: authentication state is bound to a session identifier that the application never rotates, so possessing that identifier in advance is functionally equivalent to possessing the victim's login.

Real-world delivery mechanism (not tested in this lab, described for context): in a live attack, the attacker would not need physical or DevTools access to the victim's browser. A session ID can be fixed onto a victim via a crafted link containing the session parameter, a subdomain cookie set through a related vulnerability (e.g. XSS), or by exploiting an application that accepts session IDs from URL parameters.

5.7 Weak Session IDs — Source Code Comparison

DVWA's dedicated "Weak Session IDs" module was used to compare session-token generation logic across all four security tiers, obtained directly via the module's View Source feature.

Low

```
$_SESSION['last_session_id']++;
$cookie_value = $_SESSION['last_session_id'];
setcookie("dvwaSession", $cookie_value);
```

A plain, sequential, in-memory counter starting at 0. Fully predictable — possession of any one valid session ID trivially reveals adjacent valid IDs by incrementing/decrementing.

Medium

```
$cookie_value = time();
setcookie("dvwaSession", $cookie_value);
```

The Unix timestamp at generation time, with zero added entropy. Verified independently by comparing DVWA's displayed value against the system's own date +%s output — confirming the "token" is simply the visible system clock, guessable to within a small window if the approximate login time is known.

High

```
$_SESSION['last_session_id_high']++;
$cookie_value = md5($_SESSION['last_session_id_high']);
setcookie(...);
```

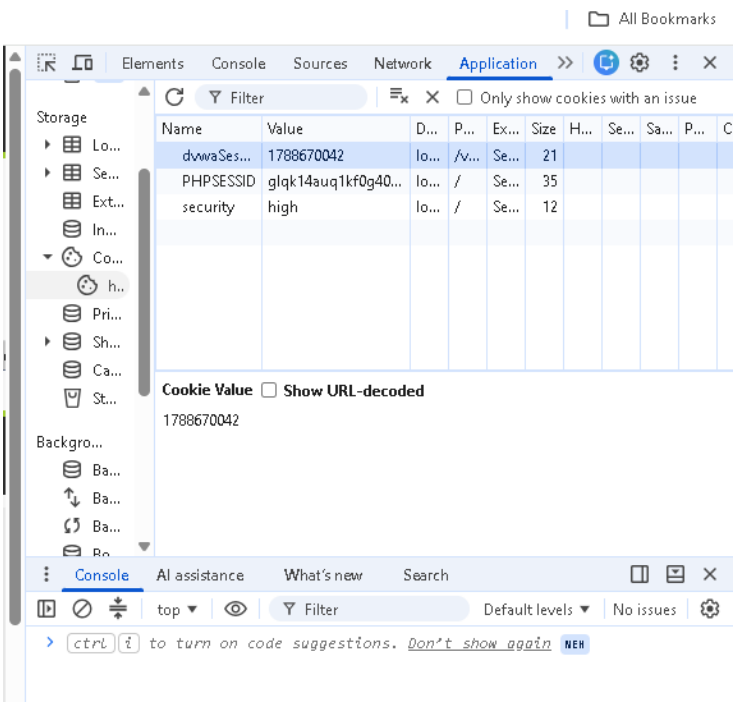

Figure 4 — DevTools Application panel confirming the security level was set to "high" and inspecting the resulting dvwaSession cookie value alongside PHPSESSID.

KEY FINDING — High applies an MD5 hash — but to the **exact same predictable incrementing counter used in Low**. Hashing a low-entropy input does not add entropy: the output looks random (a 32-character hex string) but every possible value can be precomputed in advance (e.g. md5(1), md5(2), md5(3) ...), which is precisely the mechanism a rainbow-table attack exploits. This is the single most important structural lesson of the module: **obfuscating a predictable value is not the same as making it unpredictable.**

Impossible

```
$cookie_value = sha1(mt_rand() . time() . "Impossible");
setcookie(..., true, true);
```

This tier hashes a genuinely unpredictable input (mt_rand()) combined with the timestamp and a fixed salt string — the randomness lives in the *input*, not merely the hash function, which is the structural difference that makes this tier resistant to precomputation. It also sets the secure and httponly cookie flags, unlike the other three tiers — directly consistent with a missing-flag finding from an earlier Security Misconfiguration lab on this same application.

Tier	Generation Method	Effective Entropy
Low	Sequential counter	None — trivially guessable by incrementing
Medium	Unix timestamp	None — equals the visible system clock at generation
High	MD5(sequential counter)	None — hash of the same low-entropy Low-tier input
Impossible	SHA1(random + time + salt)	High — randomness sourced from mt_rand()

6. Hydra Command Reference (All Commands Used)

The complete, ordered set of Hydra invocations from this session, for reuse and reference:

1. Initial attempt (failed — colon parsing error)

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost -s 8080 http-get-form \
"/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:F=incorrect:H=Cookie:
security=low; PHPSESSID=<value>"
```

2. Escaped colon (still failed — field-order error)

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost -s 8080 http-get-form \
"/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:F=incorrect:H=Cookie\:
security=low; PHPSESSID=<value>"
```

3. Corrected — successful crack (Low security)

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost -s 8080 http-get-form \
"/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie\: security=low;
PHPSESSID=<value>:F=incorrect"

# Result: [8080][http-get-form] host: localhost login: admin password: password
```

4. Same attack against Medium security (delay bypass)

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost -s 8080 http-get-form \
"/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie\: security=medium;
PHPSESSID=<value>:F=incorrect"
```

Flag Reference

Flag / Field	Purpose
-l admin	Fixed username to attack (single account)
-P	Path to the password wordlist to iterate over
-s 8080	Target port
http-get-form	Hydra module for form-based auth submitted via HTTP GET
username=^USER^&password;=^PASS^	Template showing where Hydra injects candidate credentials
F=	Failure condition — a response containing this string is treated as a failed login
S=	(Alternative) success condition — a response containing this string is treated as success
H=Cookie\:...	Custom HTTP header (session + security-level cookies), with the colon escaped

7. Analytical Breakdown — Why Each Attack Worked

Vulnerability	Root Cause	Contributing Factor
Credential brute force	No rate limiting, no lockout, no CAPTCHA on the login endpoint	GET-based submission also exposes credentials in the URL/history
Medium delay bypass	sleep() throttles per-request latency, not aggregate request volume	16 parallel Hydra threads absorb the delay collectively
Session fixation	Session ID is never regenerated on privilege change (pre- to post-login)	Whoever holds the ID before login automatically gains the authenticated state after
Weak/High session IDs	Token entropy is bound to the underlying input, not the output hash function	Hashing a small, enumerable input space (a counter) is fully precomputable

Connecting Thread Across All Four Exercises

Every exercise in this session traces back to the same underlying theme: **a control that looks present is not the same as a control that is effective.** A delay exists, but doesn't scale with attacker parallelism. A hash exists, but is applied to a value with no real entropy. A session mechanism exists, but never rotates its own identifier. In every case, the surface-level presence of a security measure masked the absence of the property it was supposed to guarantee.

8. Impact Assessment

- **Full account takeover via brute force** is achievable in seconds against any account using a wordlist-common password, with no attacker skill beyond running a public tool.
- **Session fixation enables complete account hijacking with zero credential knowledge** — an attacker only needs to get a victim to authenticate while holding a session ID the attacker already possesses (e.g. via a crafted link).
- **Weak or superficially-hashed session tokens allow session prediction/hijacking at scale** — an attacker could precompute the entire valid token space for the High tier in a fraction of a second and simply test each one against the application.
- **Combined**, these three weaknesses mean an attacker has multiple independent paths to full account compromise, several of which require no interaction with the victim's actual credentials at all.

9. Remediation & Fixes

Finding	Recommended Fix
No brute-force protection	Implement per-account and per-IP rate limiting plus temporary lockout after N failed attempts

Finding	Recommended Fix
Credentials sent via GET	Change the login form's method to POST to avoid credential exposure in URLs/logs/history
sleep()-only throttling	Pair (or replace) with lockout and IP-based rate limiting that account for concurrent/parallel requests
Session ID never regenerated at login	Call session_regenerate_id() immediately upon successful authentication and invalidate the prior ID
Session cookies missing Secure/HttpOnly flags	Set Secure, HttpOnly, and SameSite attributes on all session cookies (as already done correctly at the Impossible tier)
Predictable session ID generation (Low/Medium/High)	Generate tokens from a cryptographically secure random source (e.g. random_bytes()) rather than counters, timestamps, or hashes of either

10. Skills & Baseline Established

This session builds directly on prior lab work (SQLi, XSS, Security Misconfiguration, Logging & Monitoring Failures) and establishes the following new, verifiable baseline:

Category	Skill Demonstrated
Tool proficiency	Hydra command construction, flag syntax, and troubleshooting field-order/escaping errors
Attack technique	Automated credential brute forcing against a GET-based login form
Analytical skill	Recognizing that a defense's presence (delay, hash) does not guarantee its effectiveness
Offensive technique	Session fixation — manually forcing and proving cross-browser session hijack
Source review	Reading raw PHP to compare session-token generation strategies across security tiers
Browser tooling	Chrome DevTools cookie inspection/editing (Application tab), Incognito-based isolation testing
Security judgment	Distinguishing genuine entropy sources (mt_rand()) from disguised-but-predictable ones (hashed counters)

11. Conclusion & Next Steps

This session provided complete, hands-on coverage of OWASP A07:2021 across its core failure modes: weak credential enforcement, ineffective rate limiting, session fixation, and predictable session token generation. Each finding was independently reproduced and verified rather than taken on faith from documentation, consistent with the practical, verification-first methodology used throughout this lab series.

Planned next steps:

- Optional extension: DVWA's Insecure CAPTCHA module (bypassing a client-side/step-skippable CAPTCHA check).
- Publish this lab as a structured write-up (Overview → Environment → Steps → Why It Worked → Impact & Fix) in the ongoing GitHub OWASP Top 10 repository, following the same format as prior entries.
- Continue the OWASP Top 10 roadmap toward the remaining uncovered categories (A02, A04, A06, A08, A10).

End of Report — A07:2021 Identification & Authentication Failures — DVWA Lab Session